Matteo Liotta, `matteo.liotta@studenti.units.it` 28/10/2025

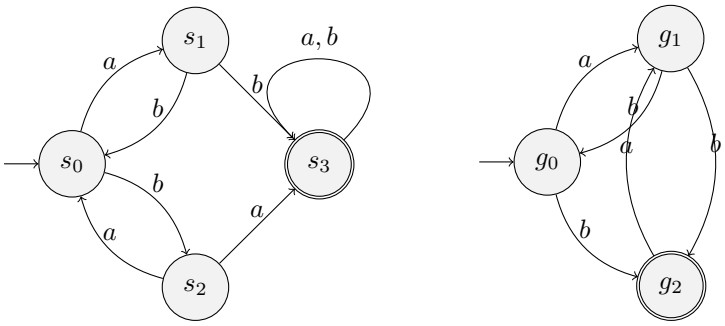
# Esercitazione 08: *Soluzione Appello 28 febbraio 2025*

***Nota***. Sono indicati in *blu* gli esercizi che non saranno trattati durante l'esercitazione, bensì lasciati come strumento aggiuntivo di esercitazione individuale. Resta comunque assoluta disponibilità per la loro risoluzione negli incontri successivi, qualora necessario.

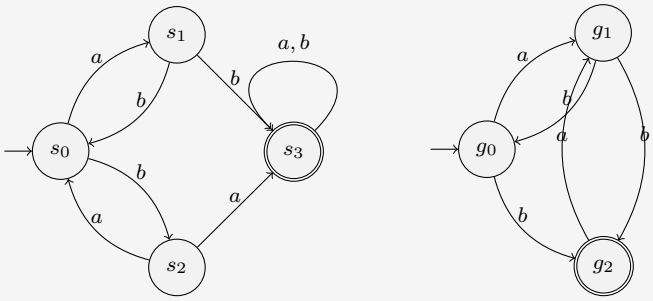
## Esercizio 1 (Punti 7)

Dati i due automi in figura

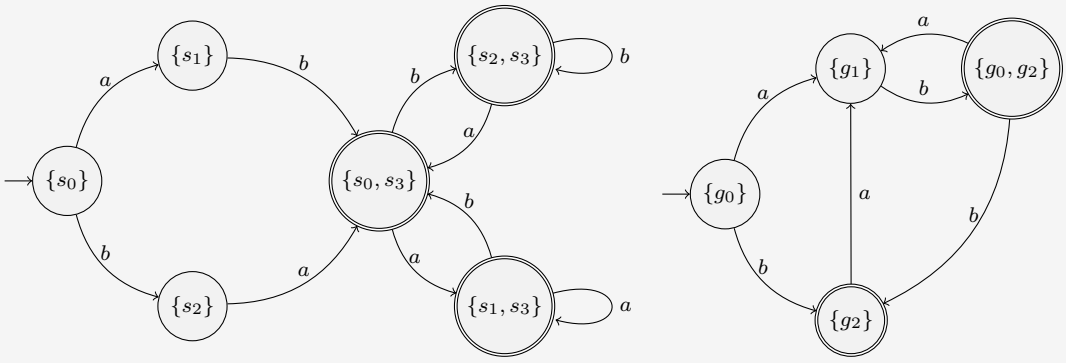
- Si costruisca l'automa per l'intersezione dei due automi;
- si costruisca l'automa per l'espressione regolare $((ab)^* \cdot (bc)^*) \cdot b^*$




*Soluzione.*
Otteniamo le determinizzazioni:
Costruiamo dapprima l'automa prodotto, in cui sono mantenuti solo gli archi comuni.

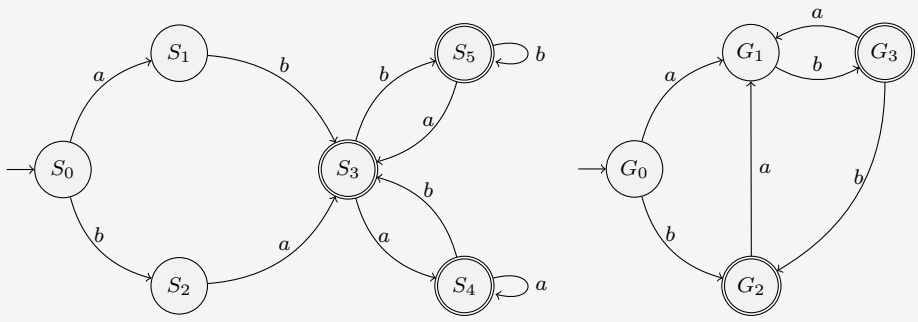



diventano :

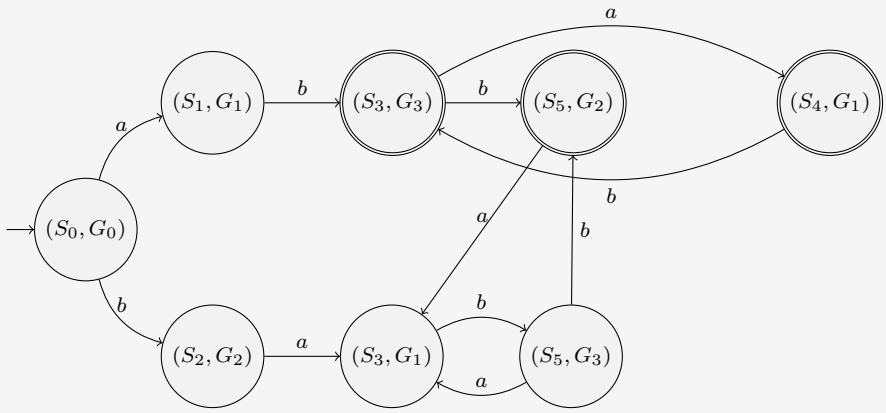


Matteo Liotta, `matteo.liotta@studenti.units.it` 28/10/2025

Per chiarezza, in quanto questi degli automi deterministici, nulla ci vieta di *rinominare gli stati*:

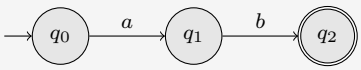



L'automa prodotto allora è il seguente.

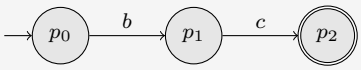



Invece, per quanto riguarda la seconda parte, possiamo scrivere l'espressione regolare sfruttando la possibilità di costruirlo inv via modulare:

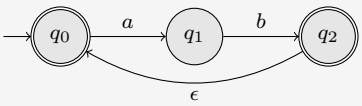
- $ab$:



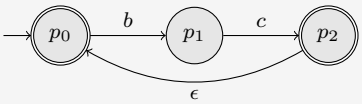

- $bc$:




- $(ab)^*$:

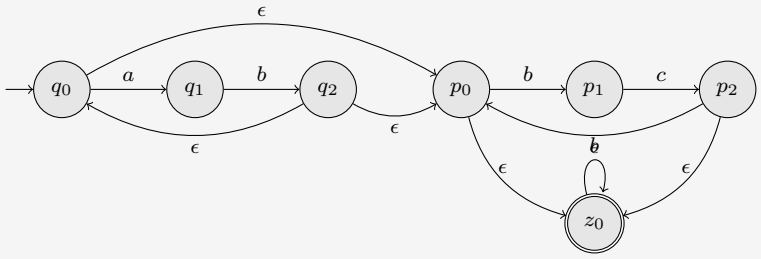



- $(bc)^*$:



Matteo Liotta, `matteo.liotta@studenti.units.it` 28/10/2025

- $((ab)^*(bc)^*)b^*$:




## Esercizio 2 (Punti 6)

Si scriva una grammatica libera dal contesto che definisce il linguaggio delle parentesi tonde correttamente accoppiate come tradizionalmente si trovano nelle espressioni aritmetiche ad esempio: $(), ((()))$, $(()())(())(())()$, *etc.*

*Soluzione.*
Per risolvere l'esercizio dovremo avere sempre $N$ parentesi chiuse ) dopo altrettante $N$ aperte (, perciò:

- Dovremo avere sempre all'inizio un (, mai )
- Possiamo utilizzare l'idea di mantenere un debito, proprio come per il linguaggio delle parole con numero di a e b uguale.

La grammatica proposta è la seguente:

```
S -> A
A -> () | (R | ϵ    ← L'idea è di avere la possibilità di mantenere un debito, che potremo saldare o aumentare
R -> )A | (RR       ← Saldiamo il debito o lo incrementiamo (unitario)
```

## Esercizio 3 (Punti 7)

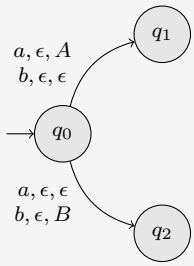
Si scriva un automa a pila che accetta il seguente linguaggio $\{w^1\sharp w^2 : w^1, w^2 \in \{a,b\}^+,$ o $|w^1|_a = 2|w^2|_a$ oppure $|w^1|_b = 2|w^2|_b\}$ (o doppio numero di a in $w^1$ rispetto a $w^2$ oppure doppio numero di $b$).

*Soluzione.*
La soluzione può sfruttare il non determinismo:

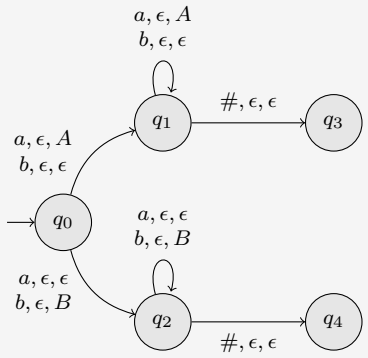
- All'inizio, possiamo leggere o $a$ o $b$, ma solo in un percorso sceglieremo di controllare se il numero di $a$ è doppio, nell'altro le $b$.
- Per via della possibilità di considerare di terminazione tutte le computazioni che portino ad uno stato finale, segue la verifica in contemporanea di ambo le possibilità.

(Si noti come la transizione etichettata con $x, AA, \epsilon$ estrae due $A$ singoli dalla pila, mentre $x, \epsilon, AA$ ne inserisce due singoli. Lo si assume per brevità.).

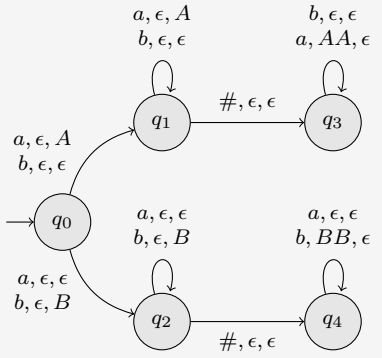
Possiamo partire con:



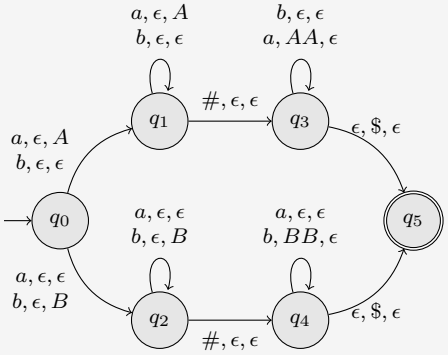
Nel precedente si mostra quindi il non determinismo anticipato precedentemente. A questo punto è possibile continuare ammettendo altre letture e raggiungendo il simbolo che separa le due parole:




Raggiunta la parola $w_2$, vediamo se il numero di $a$ è effettivamente doppio, scaricando un simbolo dalla pila per ogni lettura, ignorando l'altra lettera:




Terminata la lettura della parola, la pila deve essere vuota:




## Esercizio 4 (Punti 7)

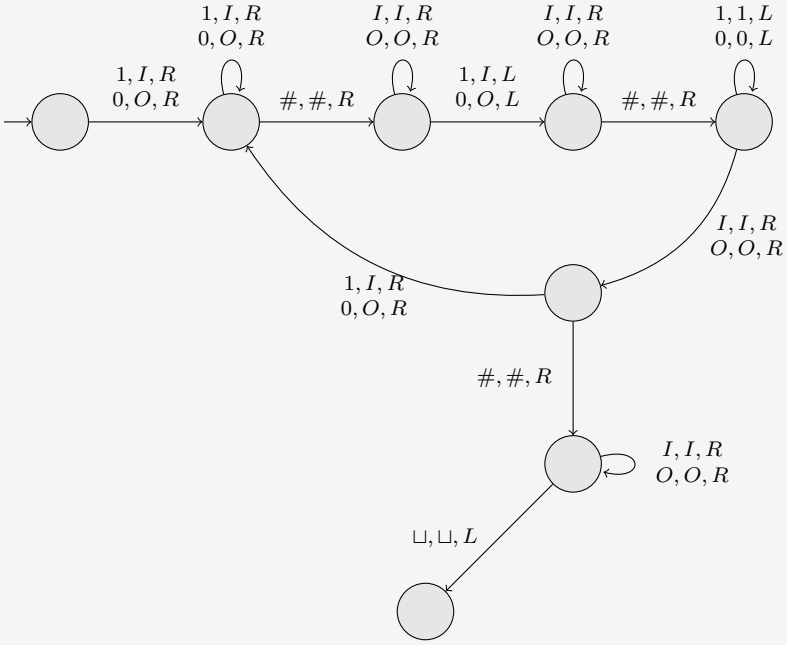
Si scriva una macchina di Turing che ha in input sul nastro tre stringhe sull'alfabeto $\{0, 1\}$ separate dal simbolo $\sharp$. La macchina di Turing accetta se le due stringhe hanno uguale lunghezza lasciando sul nastro l'AND logico delle due stringhe (AND dei bit nella medesima posizione nelle due stringhe).

Matteo Liotta, `matteo.liotta@studenti.units.it` 28/10/2025

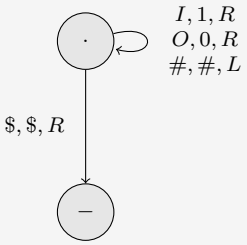
*Soluzione.*
La macchina di turing può essere costruita in via più compatta, ma per chiarezza vediamola nella via più semplice possibile: suddividendo la costruzioni in sottoparti specializzate in uno specifico task.

1. Dapprima, dobbiamo occuparci del controllo sulla lunghezza delle parole, analogamente a quanto fatto per verificare il linguaggio $a^n b^n$;
2. In seguito, ci si occupa dell'AND logico element-wise;
3. Infine, si cancella la parola dopo #.

Si noti che, per semplicità, si assume sia possibile riconoscere l'inizio del nastro, grazie al simbolo \$ scritto prima di $w_1$; l'assunzione non è problematica in quanto è sempre possibile inserirla eventualmente, facendo scorrere le parole sul nastro come passo iniziale.

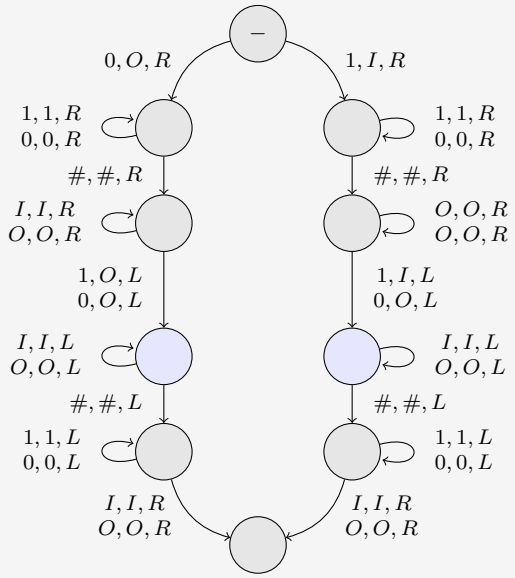


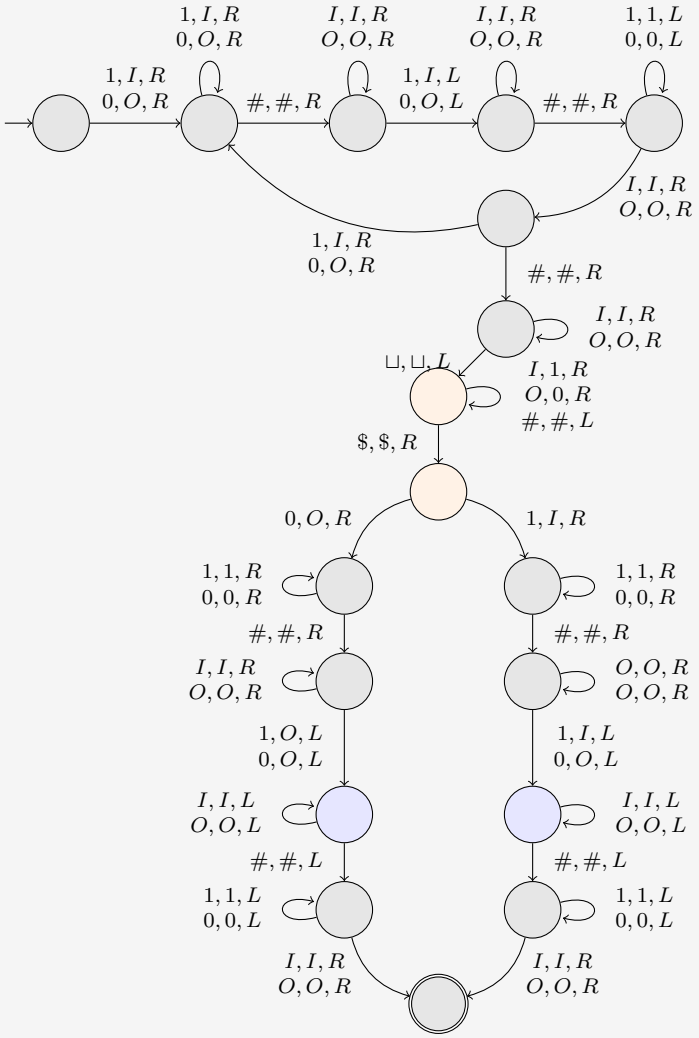

A questo punto, possiamo raggiunto l'ultimo stato, possiamo aspettarci che le parole siano state controllate e siano di uguale lunghezza. Possiamo tornare all'inizio e, facendolo, possiamo tornare sostituire i simboli $I$ con 1 e $O$ con 0.




Infine, è necessario occuparsi della gestione della porzione dell'automa specializzato nella computazione dell'AND logico tra le due parole. Dovremo, chiaramente, distinguere due casi:

- Se la prima parola ha l'$i$-esima lettera pari a 0, allora il risultato di AND sarà 0 indipendentemente dalla $i$-esima lettera di $w_2$;
- Se l'$i$-esima lettera è un 1 per $w_1$, allora il risultato di AND sarà:
  - 1, se $w_2$ ha 1 in $i$-esima posizione;
  - 0, se $w_2$ ha 0 in $i$-esima posizione




Quindi, il risultato finale è il seguente: